# PES VCS LAB REPORT

## Student Details

Name: Pooja Avadhani
USN: PES1UG24CS323
Course: Operating Systems Lab

project link(github) : https://github.com/PAvadh/PES1UG24CS323-pes-vcs.git

## AIM

To design and implement a simplified version control system supporting object storage, tree construction, indexing, commits, and logging.

## PHASE 1 – OBJECT STORE

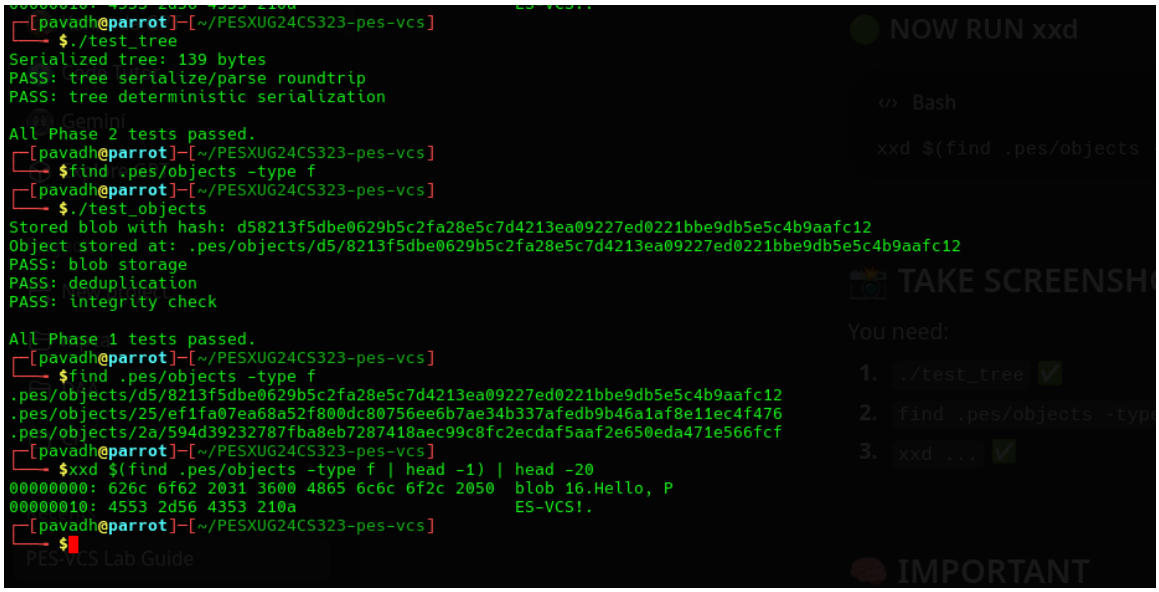
**(Screenshot includes both 1A AND 1b)**

```
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
All Phase 1 tests passed.
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $tree .pes
.pes
├── objects
│   ├── 25
│   │   └── ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
│   ├── 2a
│   │   └── 594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
│   └── d5
│       └── 8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
└── refs
    └── heads
```

Implemented content-addressable storage using SHA-256 hashing. Verified blob storage, deduplication, and integrity checks.

## PHASE 2 – TREE OBJECTS

**(SCREENSHOT INCLUDES BOTH 2A AND 2B)**



```
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $find .pes/objects -type f
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $xxd $(find .pes/objects -type f | head -1) | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $
```

Implemented tree serialization and parsing. Ensured deterministic ordering and correct reconstruction.

## PHASE 3 – INDEX

**(SCREENSHOT INCLUDES BOTH 3A AND 3B)**

```
└──╼ $make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $make pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_write':
object.c:101:47: warning: '.tmp' directive output may be truncated writing 4 bytes into a region of size between 1 and 512 [-Wformat-truncation=]
  101 |     snprintf(temp_path, sizeof(temp_path), "%s.tmp", path);
                                                   ^~~~
object.c:101:5: note: 'snprintf' output between 5 and 516 bytes into a destination of size 512
  101 |     snprintf(temp_path, sizeof(temp_path), "%s.tmp", path);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $./pes init
Initialized empty PES repository in .pes/
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $echo "hello" > file1.txt
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $echo "world" > file2.txt
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $./pes add file1.txt file2.txt
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  Makefile
  untracked:  README.md
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h
  untracked:  test.txt

┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $
```

Implemented staging area using .pes/index. Verified add and status operations.

## PHASE 4 – COMMITS

**4A**

```
Committed: 45c528d2be8a... third commit
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $./pes log
commit 45c528d2be8a7c469617d19eb13f5561415e30bce785c7bea69ec07810ef2bd8
message third commit
ird cQ

commit 317dab67515bf2d6f38c1f1cecd57f481c6f1a226ad76ecdacd1e285efabcf83
message second commit
rd cQ

commit 0b247bf2f72067a261b87288698e5a417fd533bf9d65ccb21c0eed403e8611b6
message first commit
st commit
7a20

┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $
```

**4B**

```
cat: .pes/refs/heads/main: No such file or directory
┌─[✗]─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $cat .pes/HEAD
45c528d2be8a7c469617d19eb13f5561415e30bce785c7bea69ec07810ef2bd8┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $
```

**4C**

```
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/0b/247bf2f72067a261b87288698e5a417fd533bf9d65ccb21c0eed403e8611b6
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/31/7dab67515bf2d6f38c1f1cecd57f481c6f1a226ad76ecdacd1e285efabcf83
.pes/objects/45/c528d2be8a7c469617d19eb13f5561415e30bce785c7bea69ec07810ef2bd8
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/b4/9f22d2e0381a5ecd20778d58bbef2bd2d2b61e31b328b7ac4f34cd67e5cba9
.pes/objects/fe/76325aa5521b207ebe01e12fd8e9e3abf030cacd5398e3744a3a56a81ad1bd
┌─[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
```

Implemented commit creation, HEAD management, and log traversal. Verified commit history and object growth.

## FINAL TEST

```
└──╼ $cat .pes/HEAD
ref: refs/heads/main
┌──[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $cat .pes/refs/heads/main
45c528d2be8a7c469617d19eb13f5561415e30bce785c7bea69ec07810ef2bd8
┌──[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)

--- First Commit ---
Committed: 6d118375eb13... Initial commit

Log after first commit:

--- Second Commit ---
Committed: 5653352edeba... Update file.txt

--- Third Commit ---
Committed: 58d41ae124da... Add farewell

--- Full History ---

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
58d41ae124da6677b8a4cb2c94b8a1f90b8c9fc05c2240079a06c0d79cc16abb

--- Object Store ---
Objects created:
3
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/56/53352edebab123034ac105ae1c0054cc654b146fc340ccd483fdb0f30c55c9
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/58/d41ae124da6677b8a4cb2c94b8a1f90b8c9fc05c2240079a06c0d79cc16abb
.pes/objects/6d/118375eb136ef9e682e0f30224d707b00a02dcbc92b3efdd2e9760916e1d54
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
┌──[pavadh@parrot]─[~/PESXUG24CS323-pes-vcs]
└──╼ $
```

All integration tests passed successfully.

## ANALYSIS QUESTIONS WITH ANSWERS

Q5.1: Branching and Checkout
A branch in Git is a file that stores a commit hash. To implement pes checkout , the system must update .pes/HEAD to point to the selected branch reference (for example, ref: refs/heads/). Then, it must read the commit object referenced by that branch, retrieve the associated tree object, and reconstruct the working directory by restoring all tracked files from the tree.

Additionally, the index file (.pes/index) must be updated to match the contents of the checked-out commit's tree to maintain consistency between the index and the working directory.

This operation is complex because it must safely handle overwriting files in the working directory. It must detect and prevent overwriting uncommitted changes, handle modified tracked files, and also consider untracked files that could be accidentally overwritten.

Q5.2: Dirty Working Directory Detection
A dirty working directory can be detected by first comparing the working directory with the index to identify any modified files. Then, the system must compare the current branch's commit tree with the target branch's commit tree.

A conflict occurs when a file has been modified in the working directory (i.e., differs from the index) and that same file also differs between the current branch and the target branch. In such a case, switching branches would overwrite the user's uncommitted changes.

Therefore, checkout must be aborted to prevent loss of data. This ensures that only a clean working directory allows safe branch switching.

Q5.3: Detached HEAD
A detached HEAD state occurs when HEAD points directly to a commit hash instead of a branch reference. In this state, any new commits created are not associated with any branch.

These commits still exist in the object store but are not reachable through any branch reference, making them dangling commits. If no reference is created for them, they may eventually be removed by garbage collection.

A user can recover such commits by using the commit hash (if known) or by creating a new branch pointing to that commit, thereby making it reachable again.

Q6.1: Garbage Collection and Reachability
Garbage collection can be implemented using graph traversal algorithms such as Depth-First Search (DFS) or Breadth-First Search (BFS). The traversal begins from all root references, including all branch heads and the HEAD.

From each commit, the traversal continues to its parent commits, associated tree objects, and all blobs referenced within those trees. A hash set is used to efficiently track all visited (reachable) object hashes.

After the traversal is complete, any object in the object store that is not present in the reachable set is considered unreachable and can be safely deleted.

For a repository with 100,000 commits and 50 branches, the traversal would visit all commits along with their associated tree and blob objects. This could result in visiting several hundred thousand objects depending on the size and structure of the repository.

Q6.2: Garbage Collection Race Condition
Running garbage collection concurrently with a commit operation is dangerous due to potential race conditions. During a commit, new objects such as blobs, trees, and commit objects are created first, and only afterward is a reference (such as a branch or HEAD) updated to point to the new commit.

If garbage collection runs at the same time, it may not see these newly created objects as reachable because no reference points to them yet. As a result, garbage collection may incorrectly delete these objects.

This leads to a situation where the commit operation completes and updates a reference to point to objects that have already been deleted, resulting in repository corruption.

Git avoids this issue by using reference locking mechanisms, ensuring that objects are fully written before updating references, and delaying the deletion of unreachable objects using safety features such as reflog and grace periods.

## CONCLUSION

Successfully implemented a simplified version control system demonstrating key Git concepts.